SELECT Basics — Querying Data in PostgreSQL

> ***Chapter 12 | PostgreSQL Complete Guide*** *Module: 02_SQL_Basics | Difficulty: Beginner | Estimated Reading Time: 40 minutes*

Table of Contents

Learning Objectives

By the end of this chapter, you will be able to:

- Write correct SELECT statements retrieving specific columns and all columns
- Apply WHERE clauses to filter rows using conditions and expressions
- Sort result sets with ORDER BY, including multi-column and expression sorts
- Implement cursor-style pagination using LIMIT and OFFSET
- Eliminate duplicate rows with DISTINCT and understand its performance cost

---

Theory: The SELECT Statement

SELECT is the primary read command in SQL. In PostgreSQL, every query — no matter how simple or complex — begins with SELECT (or is built on top of it).

Logical Order of Clause Evaluation

SQL clauses are written in one order but evaluated in a different order. Understanding this prevents many common errors.

```
Written order:          Evaluation order:
--------------          ----------------
SELECT                  1. FROM / JOIN
FROM                    2. WHERE
```

```
WHERE                   3. GROUP BY
GROUP BY                4. HAVING
HAVING                  5. SELECT  (aliases defined here)
ORDER BY                6. DISTINCT
LIMIT / OFFSET          7. ORDER BY
                        8. LIMIT / OFFSET
```

This is why you **cannot** reference a SELECT alias inside a WHERE clause — the alias doesn't exist yet when WHERE is evaluated.

The Relational Model and SELECT

A SELECT statement operates on sets of rows (relations). The output is itself a relation (a result set), which can be used as input to another query (subquery, CTE, view).

Basic SELECT Syntax

Full Syntax Template

```
SELECT [ ALL | DISTINCT [ ON (expression_list) ] ]
       select_expression [, ...]
FROM   table_reference [, ...]
[ WHERE condition ]
[ GROUP BY grouping_expression [, ...] ]
[ HAVING condition ]
[ ORDER BY sort_expression [ ASC | DESC ] [ NULLS FIRST | NULLS LAST ] [, ...] ]
[ LIMIT { count | ALL } ]
[ OFFSET start [ ROW | ROWS ] ]
[ FETCH { FIRST | NEXT } count { ROW | ROWS } ONLY ]
```

Selecting All Columns

```
-- Asterisk retrieves every column
SELECT * FROM employees;
```

Selecting Specific Columns

```
-- Only the columns you need
SELECT employee_id, first_name, last_name, salary
FROM   employees;
```

Column Aliases

```
-- AS keyword (optional but recommended for clarity)
SELECT first_name AS "First Name",
       last_name  AS "Last Name",
```

```
       salary     AS monthly_salary
FROM   employees;
```

Computed Columns

```
-- Expressions in the SELECT list
SELECT first_name || ' ' || last_name AS full_name,
       salary * 12                   AS annual_salary,
       ROUND(salary * 0.1, 2)        AS bonus_estimate
FROM   employees;
```

Selecting from No Table

```
-- PostgreSQL allows SELECT without FROM for expressions
SELECT 1 + 1            AS two,
       NOW()            AS current_ts,
       UPPER('hello')   AS greeting;
```

WHERE — Filtering Rows

The WHERE clause accepts any Boolean expression. Only rows where the expression evaluates to TRUE are included (rows evaluating to FALSE or NULL are excluded).

Basic Comparisons

```
-- Equality
SELECT * FROM employees WHERE department_id = 10;

-- Inequality
SELECT * FROM employees WHERE department_id <> 10;   -- ANSI standard
SELECT * FROM employees WHERE department_id != 10;   -- also valid in PostgreSQL

-- Greater/less than
SELECT * FROM employees WHERE salary > 50000;
SELECT * FROM employees WHERE hire_date <= '2020-01-01';
```

Combining Conditions

```
-- AND: both conditions must be true
SELECT * FROM employees
WHERE  department_id = 10
  AND  salary > 60000;

-- OR: either condition must be true
SELECT * FROM employees
WHERE  department_id = 10
  OR   department_id = 20;
```

```sql
-- NOT: inverts the condition
SELECT * FROM employees
WHERE  NOT (department_id = 10);
```

WHERE with Text

```sql
-- Case-sensitive equality
SELECT * FROM products WHERE category = 'Electronics';

-- Pattern matching with LIKE
SELECT * FROM customers WHERE last_name LIKE 'Smi%';

-- Case-insensitive search with ILIKE (PostgreSQL-specific)
SELECT * FROM customers WHERE email ILIKE '%@gmail.com';
```

ORDER BY — Sorting Results

Without ORDER BY, PostgreSQL makes **no guarantee** about row order. Always specify ORDER BY when order matters.

Single Column Sort

```sql
-- Ascending (default)
SELECT * FROM employees ORDER BY last_name ASC;

-- Descending
SELECT * FROM employees ORDER BY salary DESC;
```

Multi-Column Sort

```sql
-- Sort by department, then by salary descending within each department
SELECT department_id, first_name, last_name, salary
FROM   employees
ORDER BY department_id ASC,
         salary         DESC;
```

Sorting by Column Position (avoid in production)

```sql
-- Positional reference: 1 = first SELECT column
SELECT first_name, last_name, salary
FROM   employees
ORDER BY 3 DESC;   -- sorts by salary
```

Sorting by Expression

```sql
-- Sort by computed value
SELECT product_name, price, quantity,
       price * quantity AS total_value
FROM   inventory
ORDER BY price * quantity DESC;

-- Sort by string length
SELECT last_name FROM employees ORDER BY LENGTH(last_name) DESC;
```

NULL Ordering

```sql
-- NULLs sort LAST in ASC by default in PostgreSQL
-- NULLs sort FIRST in DESC by default in PostgreSQL
-- Override explicitly:
SELECT * FROM employees ORDER BY commission_pct ASC  NULLS LAST;
SELECT * FROM employees ORDER BY commission_pct DESC NULLS FIRST;
```

LIMIT and OFFSET — Pagination

LIMIT

```sql
-- Return only the first 10 rows
SELECT * FROM products ORDER BY product_id LIMIT 10;
```

OFFSET

```sql
-- Skip the first 20 rows, return the next 10 (page 3 of size 10)
SELECT * FROM products
ORDER BY product_id
LIMIT  10
OFFSET 20;
```

Page Number Formula

```
page_number (1-based):  OFFSET = (page_number - 1) * page_size
                        LIMIT  = page_size
```

FETCH FIRST (SQL Standard Alternative)

```sql
-- Equivalent to LIMIT 10 OFFSET 20 but SQL:2008 standard
SELECT * FROM products
ORDER BY product_id
OFFSET 20 ROWS
FETCH  NEXT 10 ROWS ONLY;
```

LIMIT ALL

```
-- LIMIT ALL means no limit; useful in dynamic SQL
SELECT * FROM employees ORDER BY hire_date LIMIT ALL;
```

DISTINCT — Removing Duplicates

Basic DISTINCT

```
-- Return unique department IDs
SELECT DISTINCT department_id
FROM   employees
ORDER BY department_id;
```

DISTINCT on Multiple Columns

```
-- Unique (department_id, job_id) combinations
SELECT DISTINCT department_id, job_id
FROM   employees
ORDER BY department_id, job_id;
```

DISTINCT ON (PostgreSQL-specific)

DISTINCT ON keeps only the **first** row of each group defined by the expression. Combined with ORDER BY, this is a powerful "latest record per group" pattern.

```
-- Most recent order per customer
SELECT DISTINCT ON (customer_id)
       customer_id,
       order_id,
       order_date,
       total_amount
FROM   orders
ORDER BY customer_id, order_date DESC;
```

COUNT(DISTINCT ...)

```
-- Count unique values
SELECT COUNT(DISTINCT department_id) AS unique_departments
FROM   employees;
```

ASCII Visual Diagrams

SELECT Processing Pipeline

```
 RAW TABLE DATA
      |
      v
+------------------+
|   FROM / JOIN    |  <-- Identify source rows
+------------------+
      |
      v
+------------------+
|     WHERE        |  <-- Filter rows (NULLs excluded too)
+------------------+
      |
      v
+------------------+
|    GROUP BY      |  <-- Collapse rows into groups
+------------------+
      |
      v
+------------------+
|     HAVING       |  <-- Filter groups
+------------------+
      |
      v
+------------------+
|     SELECT       |  <-- Compute output columns / aliases
+------------------+
      |
      v
+------------------+
|    DISTINCT      |  <-- Remove duplicate rows
+------------------+
      |
      v
+------------------+
|    ORDER BY      |  <-- Sort result set
+------------------+
      |
      v
+------------------+
|  LIMIT / OFFSET  |  <-- Slice the sorted result
+------------------+
      |
      v
  RESULT SET
```

Pagination Model

```
employees table (20 rows):
+----+----------+--------+
| id | name     | salary |
+----+----------+--------+
```

```
|  1 | Alice    |  70000 |
|  2 | Bob      |  65000 |
...
| 20 | Zara     |  80000 |
+----+----------+--------+

Page size = 5:

Page 1: LIMIT 5 OFFSET  0  --> rows 1-5
Page 2: LIMIT 5 OFFSET  5  --> rows 6-10
Page 3: LIMIT 5 OFFSET 10  --> rows 11-15
Page 4: LIMIT 5 OFFSET 15  --> rows 16-20
```

DISTINCT ON — First Row Per Group

```
orders table (sorted by customer_id, order_date DESC):
+-------------+----------+------------+--------+
| customer_id | order_id | order_date | amount |
+-------------+----------+------------+--------+
|           1 |      105 | 2024-11-01 |    200 |  <-- KEPT (most recent for cust 1)
|           1 |       98 | 2024-09-15 |    150 |  <-- discarded
|           2 |      110 | 2024-11-05 |    300 |  <-- KEPT (most recent for cust 2)
|           2 |      102 | 2024-08-20 |     90 |  <-- discarded
+-------------+----------+------------+--------+

DISTINCT ON (customer_id) keeps the first row per customer_id group.
```

SQL Examples

```sql
-- Example 1: Simple SELECT all columns
SELECT * FROM departments;

-- Example 2: Select specific columns with aliases
SELECT department_id   AS dept_id,
       department_name AS dept_name,
       manager_id
FROM   departments;

-- Example 3: Arithmetic in SELECT list
SELECT product_name,
       unit_price,
       units_in_stock,
       unit_price * units_in_stock AS inventory_value
FROM   products;

-- Example 4: String concatenation
SELECT first_name || ' ' || last_name AS full_name,
       LOWER(email)                   AS email
FROM   employees;
```

```sql
-- Example 5: Filter with WHERE equality
SELECT * FROM employees WHERE job_id = 'IT_PROG';

-- Example 6: Filter with WHERE range
SELECT * FROM employees
WHERE  salary BETWEEN 50000 AND 80000;

-- Example 7: Filter with LIKE pattern
SELECT * FROM customers WHERE first_name LIKE 'J%';

-- Example 8: Filter with IN list
SELECT * FROM employees WHERE department_id IN (10, 20, 30);

-- Example 9: Compound WHERE with AND + OR
SELECT * FROM products
WHERE  category = 'Electronics'
  AND  (price < 500 OR discontinued = FALSE);

-- Example 10: ORDER BY single column DESC
SELECT product_name, unit_price
FROM   products
ORDER BY unit_price DESC;

-- Example 11: ORDER BY multiple columns
SELECT last_name, first_name, salary
FROM   employees
ORDER BY last_name ASC, salary DESC;

-- Example 12: ORDER BY expression
SELECT order_id, quantity, unit_price,
       quantity * unit_price AS line_total
FROM   order_details
ORDER BY quantity * unit_price DESC;

-- Example 13: LIMIT for top-N query
SELECT product_name, unit_price
FROM   products
ORDER BY unit_price DESC
LIMIT  5;

-- Example 14: LIMIT + OFFSET pagination
SELECT product_name, unit_price
FROM   products
ORDER BY product_id
LIMIT  10 OFFSET 30;   -- Page 4 (0-indexed page 3), page size 10

-- Example 15: DISTINCT on one column
SELECT DISTINCT country FROM customers ORDER BY country;

-- Example 16: DISTINCT on multiple columns
SELECT DISTINCT ship_country, ship_region
```

```
FROM   orders
ORDER BY ship_country;

-- Example 17: DISTINCT ON (PostgreSQL-specific)
SELECT DISTINCT ON (department_id)
       department_id,
       employee_id,
       first_name,
       last_name,
       salary
FROM   employees
ORDER BY department_id, salary DESC;  -- highest earner per department

-- Example 18: NULL ordering
SELECT employee_id, commission_pct
FROM   employees
ORDER BY commission_pct ASC NULLS LAST;

-- Example 19: SELECT without FROM
SELECT CURRENT_DATE AS today,
       NOW()        AS current_timestamp,
       2 + 2        AS four;

-- Example 20: Combine WHERE, ORDER BY, and LIMIT
SELECT order_id, customer_id, order_date, freight
FROM   orders
WHERE  order_date >= '2023-01-01'
  AND  freight > 100
ORDER BY freight DESC
LIMIT  20;
```

Common Mistakes

1. **Using `SELECT *` in production code.** Retrieves all columns including large BLOBs and hidden columns; breaks if the table schema changes (column added/removed alters positional references in application code). Always list columns explicitly.

2. **Forgetting ORDER BY with LIMIT.** Without ORDER BY, the database returns rows in an arbitrary order. Adding LIMIT without ORDER BY returns a non-deterministic subset — results can change between executions.

3. **Off-by-one in OFFSET pagination.** Page 1 of a 1-based page system needs `OFFSET 0`, not `OFFSET 1`. A common bug: `OFFSET = page_number * page_size` instead of `OFFSET = (page_number - 1) * page_size`.

4. **Referencing a SELECT alias in WHERE.** Aliases are not available in WHERE because WHERE is evaluated before SELECT. Use a subquery or CTE instead:

   ```
   -- WRONG:
   SELECT salary * 12 AS annual FROM employees WHERE annual > 700000;
   -- CORRECT:
   ```

```sql
SELECT * FROM (SELECT salary * 12 AS annual FROM employees) sub WHERE annual > 700000;
```

5. **Assuming DISTINCT is free.** DISTINCT requires a sort or hash operation on all selected columns. On large tables this can be slow. Consider whether a GROUP BY or EXISTS subquery is a better design.
6. **Misunderstanding DISTINCT ON ordering.** The column(s) inside `DISTINCT ON (...)` must be the leftmost columns in the ORDER BY clause, otherwise PostgreSQL raises an error.
7. **Using `!=` vs `<>`.** Both work in PostgreSQL, but `<>` is the ANSI SQL standard. Some tools and linters flag `!=`.

Best Practices

1. **Always list columns explicitly in SELECT** — avoids surprises when table structure changes and documents what data the query actually needs.
2. **Always pair LIMIT with ORDER BY** — ensures reproducible, deterministic pagination.
3. **Use table aliases for multi-table queries** — even for single-table queries in a codebase that may grow.
4. **Prefer keyset (seek) pagination over OFFSET for large datasets** — `WHERE id > last_seen_id LIMIT 10` is O(1) per page; OFFSET is O(n) because the database must read and discard skipped rows.
5. **Use `ILIKE` for user-input searches** — avoids case-sensitivity bugs without forcing UPPER/LOWER transformations on the column (which prevent index use unless a functional index exists).
6. **Qualify ambiguous column names with table alias** — prevents confusion and errors in joins.
7. **Use `FETCH FIRST n ROWS ONLY`** in portable SQL that may run on non-PostgreSQL systems.

Performance Considerations

Index Use and WHERE Clauses

- Equality conditions on indexed columns (`WHERE id = 5`) are the fastest — the planner uses an Index Scan.
- Range conditions (`WHERE salary BETWEEN 50000 AND 80000`) can use a B-tree index if the column is indexed.
- Wrapping a column in a function (`WHERE UPPER(email) = 'A@B.COM'`) prevents index use unless a matching functional index exists.

DISTINCT Performance

DISTINCT adds a sort or hash-dedup step. If you need distinct values frequently, consider a materialized view or a separate lookup table.

LIMIT/OFFSET Performance

```
OFFSET 0       -> fast (reads first N rows)
OFFSET 10000   -> slow (reads and discards 10,000 rows first)
```

For large offsets, switch to keyset pagination:

```sql
-- Keyset pagination: much faster than OFFSET for page 1000+
SELECT * FROM orders
WHERE  order_id > 99999    -- last_id from previous page
ORDER BY order_id
LIMIT  20;
```

EXPLAIN ANALYZE

Always verify query plans for non-trivial queries:

```sql
EXPLAIN ANALYZE
SELECT * FROM employees
WHERE  department_id = 10
ORDER BY salary DESC
LIMIT  5;
```

Interview Questions

1. What is the logical order of SELECT clause evaluation, and why does it matter?
2. Can you use a SELECT alias in a WHERE clause? Why or why not?
3. What happens if you use LIMIT without ORDER BY?
4. What is the difference between DISTINCT and DISTINCT ON in PostgreSQL?
5. Why is OFFSET-based pagination slow for large page numbers, and what is the alternative?
6. How does PostgreSQL handle NULLs in ORDER BY by default?
7. What is the difference between `<>` and `!=` in PostgreSQL?
8. When would you use `SELECT 1` (selecting a literal without a FROM clause)?
9. How does DISTINCT affect query performance?
10. What does `FETCH FIRST 10 ROWS ONLY` do and how is it different from `LIMIT 10`?

Interview Answers

Q1: Logical order of SELECT clause evaluation? FROM/JOIN → WHERE → GROUP BY → HAVING → SELECT → DISTINCT → ORDER BY → LIMIT/OFFSET. It matters because: (a) WHERE cannot use SELECT aliases; (b) HAVING filters after GROUP BY; (c) ORDER BY can use SELECT aliases because it runs after SELECT.

Q2: SELECT alias in WHERE? No. WHERE is evaluated before SELECT in logical order, so aliases defined in SELECT do not yet exist. Use a subquery, CTE, or repeat the expression: `WHERE salary * 12 > 700000`.

Q3: LIMIT without ORDER BY? The database returns an arbitrary subset of rows. Results are non-deterministic and can change between executions as table statistics, parallel workers, or physical storage change. Always pair LIMIT with ORDER BY.

Q4: DISTINCT vs DISTINCT ON? `DISTINCT` deduplicates the entire result row — every column must match for two rows to be considered duplicates. `DISTINCT ON (col)` is PostgreSQL-specific and keeps the first

row of each group defined by `col` (which row is "first" is controlled by ORDER BY). It is used for "latest/top record per group" queries.

Q5: OFFSET pagination problem? OFFSET causes the database to read and discard all rows up to the offset. OFFSET 10000 reads 10,000 rows and throws them away. For high page numbers this is O(n). Keyset pagination uses `WHERE id > last_seen_id` which is O(1) per page.

Q6: NULL ordering in PostgreSQL? In ascending order, NULLs sort last. In descending order, NULLs sort first. This is the opposite of some other databases. You can override with `NULLS FIRST` or `NULLS LAST`.

Q7: `<>` vs `!=`? Both mean "not equal" in PostgreSQL. `<>` is the ANSI SQL standard form; `!=` is an alias. They are functionally identical. Use `<>` for portability.

Q8: SELECT without FROM? Used to evaluate expressions, functions, or constants: `SELECT NOW()`, `SELECT 1+1`, `SELECT md5('text')`. Also used in EXISTS subqueries: `SELECT 1` inside EXISTS is idiomatic.

Q9: DISTINCT performance? DISTINCT requires an extra sort or hash step to identify duplicates. The cost depends on the number of rows and the number of columns in the DISTINCT set. It prevents streaming result delivery and increases memory usage. At scale, consider alternatives like GROUP BY or pre-aggregated tables.

Q10: FETCH FIRST vs LIMIT? `FETCH FIRST n ROWS ONLY` is SQL:2008 standard syntax. `LIMIT n` is a PostgreSQL/MySQL extension. They are functionally equivalent in PostgreSQL. `FETCH FIRST` is preferred in portable or enterprise SQL contexts.

Hands-on Exercises

Setup:

```
CREATE TABLE products (
    product_id   SERIAL PRIMARY KEY,
    product_name VARCHAR(100),
    category     VARCHAR(50),
    unit_price   NUMERIC(10,2),
    units_in_stock INT,
    discontinued BOOLEAN DEFAULT FALSE
);

INSERT INTO products (product_name, category, unit_price, units_in_stock,
discontinued) VALUES
('Widget A',   'Electronics', 29.99,  100, FALSE),
('Widget B',   'Electronics', 49.99,   50, FALSE),
('Gadget C',   'Electronics', 199.99,  20, FALSE),
('Thingamajig','Home',         9.99,  200, FALSE),
('Doohickey',  'Home',        14.99,   75, TRUE),
('Sprocket X', 'Industrial', 149.99,   30, FALSE),
('Nut Pack',   'Industrial',   2.49,  500, FALSE),
('Bolt Pack',  'Industrial',   1.99,  600, FALSE),
('Sensor Y',   'Electronics',  89.99,  15, FALSE),
('Wrench Z',   'Industrial',  34.99,   80, FALSE);
```

Exercise 1: Retrieve the product name and total inventory value (unit_price * units_in_stock) for all non-discontinued products, sorted by total inventory value descending.

Exercise 2: Find all distinct categories in the products table, sorted alphabetically.

Exercise 3: Retrieve page 2 of products (page size = 3), ordered by unit_price ascending.

Exercise 4: Find the top 3 most expensive Electronics products that are not discontinued.

Exercise 5: Using DISTINCT ON, retrieve the cheapest product in each category.

Solutions

```sql
-- Exercise 1
SELECT product_name,
       unit_price * units_in_stock AS total_inventory_value
FROM   products
WHERE  discontinued = FALSE
ORDER BY total_inventory_value DESC;

-- Exercise 2
SELECT DISTINCT category
FROM   products
ORDER BY category ASC;

-- Exercise 3
SELECT product_id, product_name, unit_price
FROM   products
ORDER BY unit_price ASC
LIMIT  3 OFFSET 3;   -- page 2 of page_size 3

-- Exercise 4
SELECT product_name, unit_price
FROM   products
WHERE  category = 'Electronics'
  AND  discontinued = FALSE
ORDER BY unit_price DESC
LIMIT  3;

-- Exercise 5
SELECT DISTINCT ON (category)
       category,
       product_name,
       unit_price
FROM   products
ORDER BY category, unit_price ASC;
```

Advanced Notes

TABLESAMPLE (PostgreSQL-specific)

For approximate results on huge tables, sample a percentage of pages:

```
SELECT * FROM large_table TABLESAMPLE BERNOULLI(10);  -- ~10% of rows randomly
SELECT * FROM large_table TABLESAMPLE SYSTEM(1);      -- ~1% of pages (faster, less
random)
```

SELECT with FOR UPDATE / FOR SHARE

Locks selected rows for subsequent UPDATE in the same transaction:

```
SELECT * FROM inventory WHERE product_id = 5 FOR UPDATE;
-- Now safe to UPDATE inventory SET qty = qty - 1 WHERE product_id = 5;
```

Lateral Subqueries

A `LATERAL` subquery can reference columns from preceding FROM items:

```
SELECT e.employee_id, latest_review.*
FROM   employees e,
LATERAL (
    SELECT review_date, score
    FROM   performance_reviews
    WHERE  employee_id = e.employee_id
    ORDER BY review_date DESC
    LIMIT  1
) latest_review;
```

The `ALL` Keyword

`SELECT ALL` is the default (opposite of DISTINCT) and is rarely written explicitly. It exists for completeness.

Window Functions vs DISTINCT ON

DISTINCT ON is a shorthand for a common pattern, but window functions like `ROW_NUMBER()` give more flexibility:

```
-- Equivalent to DISTINCT ON but more flexible:
SELECT * FROM (
    SELECT *, ROW_NUMBER() OVER (PARTITION BY category ORDER BY unit_price) AS rn
    FROM products
) sub
WHERE rn = 1;
```

Cross-References

- **Previous:** 02_dml_commands.md — INSERT, UPDATE, DELETE, MERGE
- **Next:** 04_filtering_and_operators.md — Comparison, Logical, BETWEEN, IN, LIKE, regex
- **Related:** 05_null_handling.md — NULL semantics and IS NULL in WHERE
- **Related (03_Intermediate):** `03_Intermediate_SQL/01_joins.md` — Multi-table SELECT with JOINs

- **Related (03_Intermediate):** `03_Intermediate_SQL/02_aggregations.md` — GROUP BY and HAVING
- **Related (03_Intermediate):** `03_Intermediate_SQL/04_window_functions.md` — Window functions for ranking and pagination